

BetDoc: Comprehensive Backend Technology Stack

This document outlines the complete backend technology stack, architecture, and tooling utilized in the BetDoc platform. It provides a technical overview of the system's foundation, designed for high-concurrency, strict data validation, and real-time state synchronization.

1. Core Framework & Runtime

- **Python (3.12+)**: The foundational runtime, leveraging the latest asynchronous features (`async/await`) and strict static typing capabilities.
- **uv**: A blazingly fast, Rust-based Python package and environment manager used to ensure deterministic builds and rapid dependency resolution.
- **FastAPI**: A high-performance web framework for building APIs. It is built on Starlette (for the web parts) and Pydantic (for the data parts), providing automatic OpenAPI schema generation and heavily optimized asynchronous routing.
- **Uvicorn**: An ASGI (Asynchronous Server Gateway Interface) web server implementation used to run the FastAPI application, enabling non-blocking, high-throughput network I/O.

2. Architecture & Design Patterns

- **Domain-Driven Design (DDD)**: The codebase is strictly partitioned into DDD layers (`domain`, `application`, `infrastructure`, and `presentation`). This ensures business logic (e.g., pricing, risk, volatility) is completely decoupled from framework-specific implementation details.
- **Hexagonal Architecture (Ports & Adapters)**: The application utilizes dependency injection and abstract interfaces (Ports) to communicate with external systems (Adapters), making the core logic heavily testable and modular.

3. Data Validation & Serialization

- **Pydantic (v2)**: Utilizes a Rust-based core for extremely fast data validation, serialization, and typing. It strictly enforces the schema of incoming HTTP and WebSocket payloads, automatically rejecting malformed data before it hits the application layer.

4. Persistence & Database Layer

- **PostgreSQL**: The primary relational database system used for ACID-compliant transactional data, such as the idempotent paper-trading ledger and user wallet states.

- **SQLAlchemy (2.0+)**: The Object-Relational Mapper (ORM) used to interact with PostgreSQL. We utilize its asynchronous extensions (`asyncpg`) to prevent database queries from blocking the main event loop.
- **Alembic**: The database migration tool used in tandem with SQLAlchemy to handle schema version control and safe structural rollouts.

5. Real-Time Streaming (WebSockets)

- **FastAPI WebSockets**: Provides bi-directional, persistent TCP connections between the client and server. This is utilized for the Live Odds Broadcaster, enabling high-frequency (sub-16ms) tick updates to be pushed to the TanStack Virtual frontend without HTTP polling overhead.

6. Code Quality, Typing, & Testing

- **pytest & pytest-asyncio**: The automated testing framework. The backend maintains a rigorous suite of over 200 unit and integration tests, explicitly testing asynchronous coroutines and domain models.
- **Mypy**: A static type checker configured strictly to ensure type safety across the codebase, eliminating runtime `TypeError` classes.
- **Ruff**: An extremely fast, Rust-based linter and code formatter that replaces Flake8/Black, ensuring PEP-8 compliance and unified code style across the repository.

7. Containerization & DevOps

- **Docker & Docker Compose**: Used to containerize the FastAPI backend and PostgreSQL instances. This ensures environmental parity across local development, staging, and production servers.

BetDoc: Backend Use Cases (Simple Explanation)

This document explains what the "Backend" of BetDoc actually does. If the Frontend (the screen you look at) is the steering wheel and dashboard of a car, the Backend is the engine, the fuel lines, and the computer under the hood.

Here are the main jobs our backend performs every single day:

1. The "Paper Trading" Vault (Virtual Bank)

What it does: It acts like a highly secure virtual bank for the user.

- **The Use Case:** When a user wants to test a betting strategy without risking real money, they place a "paper bet." The backend instantly checks if they have enough virtual funds, reserves the money, deducts it from their

balance, and records a highly secure receipt.

- **Why it matters:** It is mathematically "idempotent"—which is a fancy way of saying it is impossible to accidentally charge the user twice, even if their internet lags and they click the "Bet" button ten times in a row.

2. The Live Odds Broadcaster

What it does: It acts like a live sports commentator constantly shouting out updates.

- **The Use Case:** Sports betting odds change multiple times per second. Instead of making the user refresh the web page to see new numbers, the backend opens a direct, high-speed pipeline (a WebSocket) to the user's screen. The millisecond an edge or a price changes, the backend pushes that exact number directly to the screen.
- **Why it matters:** It ensures the user never looks at stale or expired odds.

3. The Oracle (AI Assistant)

What it does: It acts as an on-demand financial advisor.

- **The Use Case:** A user clicks on a game (e.g., Chiefs vs. Eagles) and asks, *"Is this a good bet?"* The backend receives this question, looks up the exact mathematical data for that specific game, and feeds it to our AI. The backend then generates a clear, readable answer explaining exactly why there is a mathematical "edge" to be made.
- **Why it matters:** It turns raw, confusing numbers into human-readable advice instantly.

4. The Data Translator

What it does: It acts as a universal translator for chaotic data.

- **The Use Case:** Real-world sportsbooks send data in messy, completely different formats. The backend takes all this chaotic data from the outside world, cleans it up, standardizes it, and calculates the true "Win Probability."
- **Why it matters:** By the time the data reaches the user's screen, it is perfectly clean, uniform, and easy to read.

5. The Nervous System (System Health)

What it does: It acts as a heartbeat monitor.

- **The Use Case:** The backend constantly monitors itself. It tracks how many odds are "fresh" (under 30 seconds old) and calculates a system "Health Score" (e.g., 9.5 out of 10).
- **Why it matters:** If the external data feeds go down, the backend immediately knows it is looking at expired data and will suspend betting so users don't make decisions based on old information.

